PCCCS504 SOFTWARE ENGINEERING & PROJECT MANAGEMENT LAB • ACADEMIC YEAR 2026-27

IEM Admission System: MEAN Stack POC & Comprehensive Viva Guide

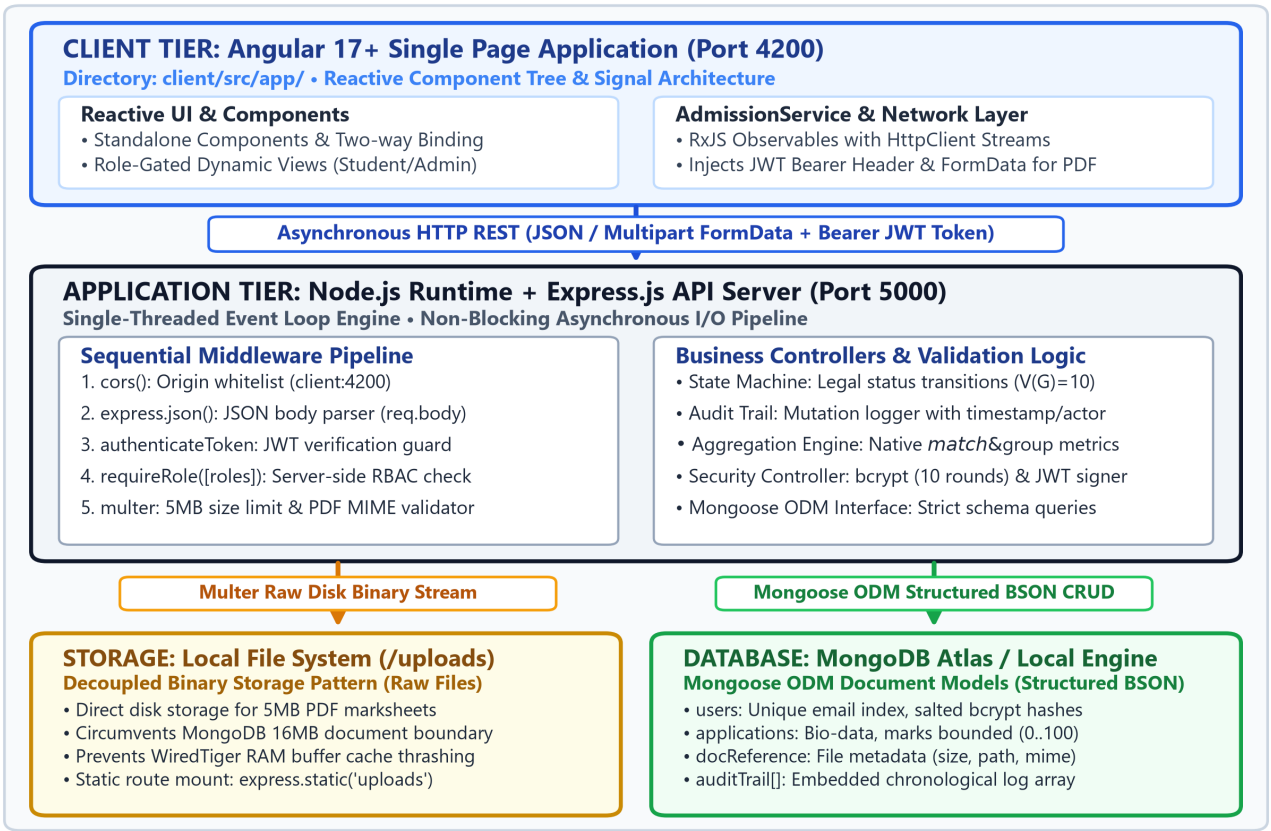
In-Depth Technical Architecture, Implementation Mechanics & Exam-Ready Viva Question Bank

Project Title:	IEM Admission System Software (MEAN Stack Proof of Concept)
Team Details:	Team Number: P4 | Team Name: Nexus | Batch: 2024-2028 (Section A, 5th Semester)
Team Members:	Susnata Maiti (A-11, Frontend) | Sreejita Dey (A-29, Backend) | Swapnil Ghosh (A-04, Database) | Debadrita Dutta (A-12, Testing)
Supervisors:	Prof. Sangeeta Sarkar, Prof. Saikat Dutta | Sponsor & Director: Dr. Satyajit Chakrabarti
POC Location:	mean_stack_equivalent/ (Client: Angular 17+ | Server: Node.js + Express + Mongoose)

PART I: COMPLETE TECHNICAL EXPLANATION OF THE MEAN STACK POC

1. Architectural Overview & Request-Response Loop

While the primary production deployment of the IEM Admission System utilizes a FastAPI (Python) backend with dual-engine persistence (MongoDB Atlas and SQLite3 fallback), the team built a complete **MEAN Stack Proof of Concept (POC)** in mean_stack_equivalent/. This POC mirrors the exact same business logic, security policies, and data models using the traditional full-stack JavaScript architecture: **MongoDB, Express.js, Angular, and Node.js**.



2. Component-by-Component Working

2.1 M — MongoDB & Mongoose Object Data Modeling (ODM)

MongoDB operates as the document database storing JSON-like BSON documents. Mongoose provides a schema-based solution to model application data, complete with validation rules and middleware:

• **User Model (models/User.js):** Manages authentication accounts. Enforces unique, lowercase email indexing and restricts the role field strictly to an enumeration: ['student', 'admin', 'director']. Passwords are saved strictly as salted cryptographic hashes; plaintext passwords never reach the database.

• **Application Model (models/Application.js):** Stores complete applicant submissions. Marks (Class 10 and 12) are bounded by schema-level validators (min: 0, max: 100). Implements a subdocument schema (AuditLogSchema) embedded as an array (auditTrail: [AuditLogSchema]) to log every status mutation with author and timestamp.

2.2 E — Express.js Routing & Middleware Pipeline

Express serves as the routing framework mounted on top of Node.js. It organizes incoming HTTP traffic through a sequential middleware pipeline:

• **Global Middleware:** cors() permits cross-origin requests from the Angular client, while express.json() parses JSON payloads into req.body.

• **Authentication Middleware (authenticateToken):** Extracts the Bearer <token> from the HTTP Authorization header and cryptographically validates it using jsonwebtoken. On failure, it immediately terminates the request with HTTP 401 Unauthorized.

• **Server-Side Role Guard (requireRole):** Inspects the verified role in req.user.role. If a student attempts to invoke administrative endpoints (such as metrics or status mutation), Express halts execution with HTTP 403 Forbidden.

2.3 A — Angular 17+ Client-Side Single Page Application

Angular provides the component-driven frontend architecture, maintaining client state without full-page browser reloads:

• **Service Layer (admission.service.ts):** Uses Angular's Dependency Injection (DI) to inject HttpClient. All network interactions return **RxJS Observable<any>** streams. The service automatically extracts JWT tokens from browser localStorage and attaches them to outbound request headers.

• **Component Layer (app.component.ts):** Subscribes to service observables, binds form values using two-way data binding, and constructs native FormData objects for binary file uploads.

2.4 N — Node.js Asynchronous Runtime

Node.js runs the V8 JavaScript engine over an asynchronous, single-threaded Event Loop. All heavy I/O tasks—such as Multer writing PDF streams to disk, bcrypt running CPU-intensive key derivation, and Mongoose querying MongoDB over TCP sockets—are dispatched to libuv's thread pool. This guarantees that concurrent applicant requests are handled without blocking.

3. Implementation of the 4 Core Project Modules

The MEAN POC implements the exact four functional modules defined in the Project Charter and SRS baseline:

Module 1: Document Upload (Decoupled Reference Pattern)

Rather than storing large PDF binaries directly in MongoDB documents (which risks hitting MongoDB's 16MB document limit and thrashing RAM cache), the POC implements a Decoupled Reference Pattern: Multer streams the PDF directly to the filesystem at server/uploads/ with an enforced 5MB ceiling and MIME check. Only lightweight metadata (filename, filepath, size) is saved into MongoDB under docReference.

Module 2: Auth & Role-Based Access Control (RBAC)

Passwords are encrypted using bcryptjs with an adaptive work factor (10 salt rounds). Valid logins receive a signed JSON Web Token encoded with user ID and role. While Angular uses client-side route guards for user experience, the backend requireRole() middleware strictly enforces server-side role isolation on all sensitive routes.

Module 3: Finite-State Machine & Audit Trail Logging

State transitions follow strict business rules evaluated by validateTransition(): Draft can only transition to Submitted; Submitted can transition to Under Review, or directly to Rejected (the latter strictly requiring explanatory remarks); terminal states (Accepted / Rejected) cannot be modified. Every transition pushes a record to the embedded auditTrail array.

Module 4: Admin Metrics Aggregation Pipeline

Instead of loading thousands of documents into Node.js memory and looping, the POC leverages MongoDB's native Aggregation Framework: $match filters out unsubmitted draft records, and $group computes real-time counts by status and department directly inside the database engine, returning a tiny ~100-byte summary to the dashboard.

4. Architectural Comparison: MEAN Stack POC vs. Production FastAPI Stack

Architectural Dimension	Production Implementation	MEAN Stack POC Equivalent
Backend Runtime	Python 3.14 + FastAPI (ASGI Coroutines)	Node.js 18+ + Express.js (Single Thread + libuv)
Type Safety & Validation	Pydantic Schemas (BaseModel) with Python type hints	Mongoose Schema validation + manual route checks
Database Architecture	Dual-Engine: MongoDB Atlas Cloud + SQLite fallback	MongoDB Atlas / Local MongoDB via Mongoose ODM
DNS SRV Resilience	Auto-injected public DNS resolvers (8.8.8.8, 1.1.1.1)	Standard Node.js dns.lookup / system resolver
Client Framework	Responsive Vanilla JS Single Page Application (SPA)	Angular 17+ (TypeScript, RxJS Observables, DI)
File Stream Handling	python-multipart streaming to backend/uploads/	multer diskStorage middleware to server/uploads/
Password Cryptography	PBKDF2-HMAC-SHA256 (600,000 hashing rounds)	bcryptjs (Blowfish adaptive hashing with salt rounds)

PART II: COMPREHENSIVE VIVA VOCE QUESTION BANK & ANSWERS

This question bank is structured across the primary engineering domains of the MEAN Stack POC. Each answer is calibrated for technical precision during laboratory vivas and external examinations:

CATEGORY A: CORE MEAN STACK ARCHITECTURE & FUNDAMENTALS

Q1: What does each letter in the MEAN stack stand for, and how does data flow end-to-end?

Answer: MEAN stands for **MongoDB**, **Express.js**, **Angular**, and **Node.js**.
The end-to-end request-response lifecycle operates in 6 distinct steps:
1. *User Action in Angular:* A user interacts with an Angular template component. The component delegates to an injected service (AdmissionService).
2. *HTTP Request Dispatch:* The service issues an asynchronous HTTP request (e.g. POST /api/auth/login) over an RxJS Observable, carrying JSON and an Authorization: Bearer <token> header.
3. *Node.js Event Handling:* Node.js receives the incoming TCP socket connection and delegates the request to the Express.js routing pipeline.
4. *Express Middleware & Business Logic:* Express filters the request through middleware (CORS, JSON parser, JWT authentication, and RBAC role validation) before invoking the controller.
5. *Mongoose & MongoDB Interaction:* The controller invokes Mongoose ODM methods (e.g., User.findOne() or Application.aggregate()) communicating with MongoDB over the binary wire protocol.
6. *Response & DOM Update:* Express serializes the result into an HTTP JSON response with standard status codes. Angular's observable subscriber receives the payload and updates component state, causing Angular's change detector to refresh the DOM without reloading.

Q2: Why is Node.js called 'single-threaded' if it handles thousands of concurrent requests simultaneously?

Answer: Node.js executes JavaScript on a single thread known as the **Event Loop** (powered by Google's V8 engine). However, Node.js offloads all non-blocking I/O operations—such as disk reads/writes via Multer, cryptographic key derivations via bcrypt, and MongoDB TCP sockets—to **libuv's internal C++ thread pool** (defaulting to 4 worker threads). When an I/O operation finishes, libuv queues its callback on the Event Loop's task queue. This asynchronous, non-blocking architecture allows Node.js to achieve high concurrency without the heavy RAM overhead of creating a dedicated OS thread per client request.

Q3: What is the architectural difference between Angular and React, and why was Angular chosen for the MEAN POC?

Answer: Angular is a complete, opinionated, battery-included **framework** written in TypeScript, featuring built-in routing, form validation, dependency injection, and HTTP services out of the box. In contrast, React is a minimalist **library** focused solely on the view layer, requiring third-party packages for routing, state, and HTTP.
Angular was chosen for the MEAN POC because its structured architectural pattern (Component-Service-Module hierarchy) enforces clean separation of concerns, robust TypeScript typing, and adherence to enterprise software engineering design principles.

CATEGORY B: MONGODB, MONGOOSE & DATA STORAGE DESIGN

Q4: Why did you store uploaded PDF marksheets on the filesystem instead of directly inside MongoDB?

Answer: Storing raw binary files inside MongoDB documents (as BSON binary buffers or GridFS chunks) creates severe bottlenecks:
1. *16MB BSON Limit:* MongoDB enforces a strict 16MB document size ceiling. Multiple high-resolution student marksheets risk hitting this limit.
2. *Memory & Cache Thrashing:* MongoDB caches active working sets in RAM via WiredTiger. Storing large binary chunks in memory evicts critical document indices and query data.
3. *Stateless File Streaming:* Files on disk can be streamed directly via static file servers (express.static) or CDNs with HTTP caching headers, bypassing database overhead entirely.
Therefore, we implemented the **Decoupled Reference Pattern**: physical PDFs reside in server/uploads/, and only lightweight metadata references (filename, filepath, size) are stored in MongoDB.

Q5: What is the difference between Embedding and Referencing in MongoDB, and how are both used in your POC?

Answer:
• **Embedding (Denormalization):** Child documents are nested inside an array within the parent document. Used for bounded 1:N relationships where child data is always queried with the parent.
POC Usage: The auditTrail array is embedded directly inside ApplicationSchema because audit logs are always read with the application and have a strictly bounded lifecycle.
• **Referencing (Normalization):** Storing an ObjectId pointing to another collection. Used for independent entities or to prevent unbounded document growth.
POC Usage: ApplicationSchema references the user record via userId: { type: ObjectId, ref: 'User' }, linking the candidate profile to authentication credentials without data duplication.

Q6: Explain how the MongoDB Aggregation Pipeline works in your /admin/metrics endpoint. Why not use find() and JavaScript reduce()?

Answer: The metrics endpoint executes a native MongoDB Aggregation Pipeline:
Application.aggregate([{ $match: { status: { $ne: 'Draft' } } }, { $group: { _id: '$status', count: { $sum: 1 } } }])
• *Pipeline Mechanism:* $match filters out draft records at the database engine level using indexes. The filtered stream passes to $group, which groups documents by categorical keys and tallies counts using native C++ accumulators.
• *Why not find() + reduce():* If an institution receives 50,000 applications, find() would serialize 50,000 JSON documents across the network, consuming hundreds of megabytes of Node.js RAM and blocking the Event Loop. The Aggregation Pipeline performs all computations inside the database engine and returns only 4 small summary objects (~100 bytes).

CATEGORY C: EXPRESS.JS, MIDDLEWARE & SECURITY

Q7: What is middleware in Express.js, and what happens if you forget to call next()?

Answer: In Express, middleware functions have access to the request object (req), response object (res), and the next middleware in the execution chain (next). Middleware can execute logic, modify request/response properties, end the cycle via res.json(), or pass control forward by invoking next().
If a middleware function neither sends a response nor calls next(), the request will **hang indefinitely** until the client socket times out.

Q8: How does your JWT authentication work, and why is client-side role checking insufficient for security?

Answer:
1. *JWT Workflow:* Upon valid login, Express issues a signed JSON Web Token consisting of three Base64URL-encoded segments: Header.Payload.Signature. The signature is encrypted with a server-side secret using HMAC-SHA256. Subsequent requests transmit this token in the Authorization: Bearer header, verified statelessly by authenticateToken.
2. *Why Client-Side Checks are Insufficient:* Client-side JavaScript running in the browser is completely under the user's control. A student can open Chrome DevTools, modify JavaScript variables, or send requests directly using Postman or curl. Client-side checks are merely a UI convenience. True security is established exclusively by server-side middleware (requireRole(['admin'])) verifying cryptographic tokens on every API endpoint.

Q9: What is the difference between bcrypt and standard hashing algorithms like SHA-256 or MD5?

Answer: SHA-256 and MD5 are general-purpose hash functions designed to be extremely fast for checksums. Because they are fast, attackers with GPUs can compute billions of guesses per second in offline dictionary attacks.
In contrast, **bcrypt** is an adaptive, one-way cryptographic hash algorithm based on the Blowfish cipher. It includes:
1. *Automatic Salting:* Appends a unique 16-byte random salt to prevent Rainbow Table attacks.
2. *Configurable Work Factor:* Developers can configure the cost factor ($2^{10} = 1024$ rounds in our POC). As hardware speeds advance, the cost factor can be increased to deliberately slow down computation, making brute-force attacks computationally infeasible.

CATEGORY D: ANGULAR SERVICES, RXJS & FINITE-STATE MACHINE

Q10: What is an RxJS Observable, and how does it differ from a standard JavaScript Promise?

Answer: In Angular's AdmissionService, all HTTP requests return Observable<any> rather than Promises.
• *Emission:* A Promise emits a single value and resolves or rejects; an Observable can emit multiple values over time (a stream).
• *Execution:* Promises are **eager** (they execute immediately upon creation); Observables are **lazy** (they do not execute until a subscriber calls .subscribe()).
• *Cancellation:* Promises cannot be cancelled once started; Observables can be cancelled at any time by calling .unsubscribe().
• *Transformation:* Observables support a rich suite of functional reactive operators (map, filter, catchError, debounceTime).

Q11: Explain the state transition logic in validateTransition. What is Cyclomatic Complexity, and how does it apply here?

Answer:
1. *Transition Logic:* Applications start in Draft, transition to Submitted, then to Under Review, and finally to terminal states (Accepted or Rejected). Direct rejection from 'Submitted' strictly requires non-empty remarks. Terminal states cannot be altered.
2. *McCabe's Cyclomatic Complexity:* Defined as $V(G) = E - N + 2P$, where E is edges, N is nodes, and P is connected components ($P=1$). In our transition function, there are 9 decision predicates, yielding $V(G) = 9 + 1 = 10$. This proves mathematically that exactly **10 linearly independent basis paths** must be tested to ensure 100% statement and branch coverage.

Q12: Why did your team deploy FastAPI for production instead of this MEAN Stack POC?

Answer: While the MEAN stack POC is fully functional, FastAPI was selected for production due to three architectural advantages:
1. *Dual-Engine Database Fallback:* FastAPI seamlessly incorporated our intelligent persistence router, automatically injecting public DNS (8.8.8.8) and failing over to local SQLite3 (iem_admission.db) in <4000ms during campus network outages.
2. *Strict Pydantic Type Validation:* Python type annotations validate incoming JSON schemas and serialize responses with zero runtime type ambiguity.
3. *Interactive OpenAPI Generation:* FastAPI automatically generates live Swagger UI documentation (/docs) directly from source code without third-party plugins.
4. *Edge Serverless Cold-Start:* On Vercel's serverless edge, Python runtime handlers boot with minimal latency compared to full Angular CLI compilations.

STRATEGIC VIVA VOCE TIPS FOR SCORING MAXIMUM MARKS

Lead with Architectural Layers:	Always explain features through both client and server layers (e.g. *'On the client, Angular builds a FormData payload; on the server, Express uses multer to stream to disk while Mongoose records only the metadata reference'*).
Emphasize Server-Side Authority:	Highlight that client-side validations are purely for user experience (UX), while true security is enforced strictly on the server by JWT verification and database schemas.
Cite Exact Quantitative Metrics:	Impress examiners by citing concrete numbers: the **5MB** PDF upload limit, **10 salt rounds** in bcrypt, **$V(G) = 10$** basis paths in the state machine, and the **16MB** BSON document limit.